

机器学习算法对比实验报告

实验题目：多种分类算法的实现与比较
学生姓名：张驰
学号：23070215
专业班级：230702
指导教师：梁音
完成时间：2026 年 4 月

机器学习算法对比实验报告1

机器学习算法对比实验报告2

摘要..... 2

1.实验环境与配置..... 3

2.算法原理与代码实现..... 3

2.1 数据读取与统一预处理.....3

2.2 逻辑回归原理与手写实现.....4

2.3 线性SVM 原理与手写实现.....5

2.4 决策树原理与手写实现.....6

2.5 sklearn 对照实验设计.....7

2.6 扩展模型选择依据.....7

2.7 统一Benchmark 计时方法.....8

2.8 三分类任务扩展设计.....8

2.9 相关研究与扩展实验依据.....9

2.10 扩展模型算法原理补充.....10

2.11 可视化分析设计.....10

3.实验结果与对比分析.....10

3.1 核心算法手写实现与sklearn 对照.....10

3.2 扩展模型比较分析.....12

3.3 原始三分类任务扩展实验.....14

3.4 噪声扰动鲁棒性分析.....15

3.5 特征消融实验分析.....16

3.6 混淆矩阵结果分析.....17

3.7 扩展实验综合分析.....18

3.8 深度学习实现方式对比实验.....19

3.9 深度学习模型训练过程分析.....20

3.10 深度学习模型鲁棒性与复杂度分析.....21

3.11 样本效率与训练规模分析.....23

3.12 概率校准与可靠性分析.....24

3.13 数据分布与决策区域可视化.....26

3.14 线性判别能力与系数分析.....28

3.15 集成模型特征重要性分析.....30

4.实验总结31

5.参考文献32

机器学习算法对比实验报告

摘要

我围绕 Iris 数据集完成了多种分类算法的实现与比较。实验一开始以课程基础要求为出发点，对逻辑回归、线性支持向量机和决策树进行了手写实现，并与 `sklearn` 成熟库版本进行了准确率和时间效率对照。在此基础上，我又继续扩展了高斯朴素贝叶斯、随机森林、AdaBoost、Gradient Boosting、MLP 神经网络以及 PyTorch 深度多层感知机等方法，并把实验进一步扩展到原始三分类任务、噪声鲁棒性分析、特征消融分析、样本效率分析、概率校准分析和温度缩放分析。

从结果上看，手写模型在当前二分类任务上能够达到与成熟库相同的分类准确率，但训练效率明显弱于 `sklearn` 实现。进一步扩展后，我发现不同方法虽然在简单任务上都能取得较高准确率，但在稳定性、训练开销、抗噪声能力、样本利用效率和概率可靠性方面仍然存在明显差异。尤其是在小样本表格任务上，传统树模型和概率模型往往表现得更加稳定，而更复杂的深度模型并不一定能够带来更高收益。通过加入决策区域、ROC 曲线、线性系数和集成模型特征重要性等可视化分析，我对各类算法的行为特点也有了更直观的理解。

关键词： 机器学习；分类算法；逻辑回归；支持向量机；决策树；深度学习；表格数据

一开始，这个实验其实只要求调用 `sklearn` 跑出三种常见分类算法的结果，完成基本训练和测试就可以了。但我在做的时候觉得，如果只是把现成接口调通，整个实验更像是“把代码跑通了”，而不是“真的理解了算法”。所以在完成基础要求之后，我又把逻辑回归、线性 SVM 和决策树重新手写了一遍，并继续补充了朴素贝叶斯、神经网络和集成学习等方法做横向比较。

这样处理以后，实验不再只是简单地把几个模型跑一遍，而是形成了一个相对完整的比较框架。前一部分主要用于验证手写实现的正确性，后一部分则用于比较不同方法在同一任务上的表现。整个项目最终覆盖了 8 种方法思路、11 个具体实现，并进一步补充了三分类扩展、稳定性分析、抗噪声分析和特征消融分析，使实验结论不再局限于单次准确率比较。

为了让扩展部分具有更明确的研究依据，本实验还参考了近年来关于表格数据学习的代表性工作。已有研究指出，传统树模型在中小规模表格数据上仍然具有很强的竞争力，而深度学习模型并不存在对所有任务都稳定占优的结论；同时，现代神经网络的概率输出还可能出现明显的过度自信现象。因此，在已有准确率和时间对比之外，后续实验又增加了样本效率、概率校准和温度缩放等分析维度，使比较范围从“分类结果是否正确”进一步扩展到“模型需要多少数据”和“模型给出的概率是否可信”。

1. 实验环境与配置

本实验在 Windows 环境下完成，命令行工具使用的是 PowerShell，Python 版本为 3.13.5。数值计算主要依赖 NumPy 2.1.3，对照和扩展实验主要依赖 scikit-learn 1.6.1，图表绘制使用 matplotlib，深度学习扩展实验则使用 PyTorch 2.8.0+cpu。项目整体结构比较清晰，main.py 负责统一组织基础实验流程，data_utils.py 负责数据读取和预处理，benchmark_utils.py 负责计时，models 目录中分别放置了三种手写模型、一组 sklearn 模型工厂函数和 PyTorch 多层感知机实现。此外，我还单独编写了 advanced_analysis.py、deep_learning_analysis.py 和 research_visualizations.py，分别用于完成三类扩展分析、深度学习实现方式与概率校准实验，以及决策边界、ROC 曲线和特征重要性等可视化分析，并将结果保存到 results 目录中。

如果从零开始搭建环境，只需要在项目根目录执行以下命令：

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install numpy scikit-learn matplotlib
python main.py
```

这次实验使用的是经典的 Iris 数据集，不过没有直接保留原始三分类任务，而是只取了前两类样本，把问题转成更适合入门分析的二分类问题。这样做之后，线性模型的分类边界会更清楚，决策树的分裂逻辑也更容易观察。数据按 8:2 划分训练集和测试集，并固定 random_state=42，目的是保证不同方法之间比较时具备统一输入条件。

2. 算法原理与代码实现

这部分我不想只是贴很多代码，因为如果只讲实现细节，内容会很像代码注释。相比之下，我更想先说清楚每个模型到底是在怎么分类，再去对应的手写实现为什么要这样写。这样一来，代码和算法含义就能对应起来，整篇内容的逻辑也会更顺一些。

2.1 数据读取与统一预处理

整个实验的起点，是先把 iris.data 文件解析成模型可以接受的数值矩阵。这里我没有继续使用 sklearn.datasets.load_iris()，而是手动读取每一行数据，并把字符串标签映射成整数，再去掉第三类样本，只保留二分类所需要的数据。

```
def load_iris_binary(path):
    label_map = {
        "Iris-setosa": 0,
        "Iris-versicolor": 1,
        "Iris-virginica": 2,
    }
    features = []
```

```

labels = []

with open(path, "r", encoding="utf-8") as file:
    for line in file:
        line = line.strip()
        if not line:
            continue

        parts = line.split(",")
        features.append([float(value) for value in parts[:4]])
        labels.append(label_map[parts[4]])

X = np.array(features, dtype=float)
y = np.array(labels, dtype=int)
mask = y != 2
return X[mask], y[mask]

```

这段代码虽然不长，但它决定了后面整个实验的风格。因为从这里开始，数据读取、数据划分、标准化和准确率计算都是自己控制的，而不是完全交给外部工具。这样做最大的好处就是，后面分析模型的时候，很多细节都看得更清楚。

2.2 逻辑回归原理与手写实现

逻辑回归虽然名字里有“回归”，但在这个实验里它本质上是一个线性分类模型。它先对输入特征做线性组合，再通过 **sigmoid** 函数把结果压缩到 0 到 1 之间，把这个值解释为样本属于某一类的概率。换句话说，逻辑回归不是在生硬地做“二选一”，而是在用一条连续的概率曲线逼近离散类别边界。它的基本形式可以写成 $p(y = 1|x) = \sigma(w^T x + b)$ ，其中 $\sigma(z) = \frac{1}{1+e^{-z}}$ 。训练时实际上是在最小化交叉熵损失，也就是让模型输出的概率尽可能贴近真实标签[5][6]。

手写实现时，最核心的部分是梯度下降。参数先初始化为 0，然后根据预测误差不断更新权重和偏置。

```

class LogisticRegressionScratch:
    def __init__(self, learning_rate=0.1, n_iters=3000):
        self.learning_rate = learning_rate
        self.n_iters = n_iters
        self.weights = None
        self.bias = 0.0

    @staticmethod
    def _sigmoid(x):
        x = np.clip(x, -500, 500)
        return 1.0 / (1.0 + np.exp(-x))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)

```

```

self.bias = 0.0

for _ in range(self.n_iters):
    linear_output = X @ self.weights + self.bias
    y_pred = self._sigmoid(linear_output)
    dw = (X.T @ (y_pred - y)) / n_samples
    db = np.mean(y_pred - y)
    self.weights -= self.learning_rate * dw
    self.bias -= self.learning_rate * db

```

真正把它手写出来之后，我对逻辑回归的理解比以前具体了很多。以前更多只是知道“它能做二分类”，现在会更自然地想到，它其实是在用一条连续的概率曲线去逼近真实标签，所以它的输出不是生硬的 0 和 1，而是带有概率意义的结果。这种感觉单靠看 API 是很难有的，还是得自己把参数更新过程走一遍，理解才会更扎实。

2.3 线性 SVM 原理与手写实现

如果说逻辑回归更像是在估计概率，那么线性 SVM 的思路就明显更偏向“拉开间隔”。它关注的不是样本到底属于某一类的概率是多少，而是两类样本之间能不能被一条边界尽量稳地分开。它希望找到的不仅是一条能分开的直线，更是一条让两边样本离边界都尽量远的分界面。

在手写实现时，我采用的是 hinge loss 的次梯度下降版本。训练时先把标签从 {0, 1} 转换成 {-1, 1}，然后只对那些落在间隔内部、或者已经被分错的样本施加更明显的更新。

```

class LinearSVMScratch:
    def __init__(self, learning_rate=0.01, n_iters=3000, c=1.0):
        self.learning_rate = learning_rate
        self.n_iters = n_iters
        self.c = c
        self.weights = None
        self.bias = 0.0

    def fit(self, X, y):
        y_transformed = np.where(y == 0, -1, 1)
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0.0

        for _ in range(self.n_iters):
            margins = y_transformed * (X @ self.weights + self.bias)
            active = margins < 1
            dw = self.weights.copy()
            db = 0.0

            if np.any(active):

```

```

        dw -= self.c * (X[active].T @ y_transformed[active]) /
n_samples
        db = -self.c * np.sum(y_transformed[active]) / n_sample
s

self.weights -= self.learning_rate * dw
self.bias -= self.learning_rate * db

```

自己写完这一段之后，我对 SVM 的印象就不再只是“又一个分类器”了，而是更明确地把它理解成一种很强调边界稳定性的模型。逻辑回归是在不断修正概率，SVM 更是在想办法把两类样本之间的间隔拉大。它对应的优化目标可以写成 $\frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i(w^T x_i + b))$ ，前一项控制边界复杂度，后一项惩罚落在间隔内部或被分错的样本[7]。虽然这里实现的是简化的线性版本，但“最大间隔”这个概念在手写之后会变得特别直观。

2.4 决策树原理与手写实现

决策树和前两个线性模型很不一样。它不直接学习一条整体边界，而是在不断追问：某个特征是不是小于某个阈值，如果是就走左边，不是就走右边。一路判断下去，最后由叶子节点给出类别。相比线性模型，决策树更像一套层层筛选的规则系统。

这次手写决策树时，我使用的是基尼指数来衡量划分效果。如果一次分裂之后，左右两边的数据类别都更加单一，那么这次划分就是好的。实现时需要遍历所有特征，枚举候选阈值，计算每次划分对应的加权基尼指数，再选出当前最优分裂点。

```

def _best_split(self, X, y):
    best_gini = float("inf")
    best_feature_index = None
    best_threshold = None
    n_features = X.shape[1]

    for feature_index in range(n_features):
        values = np.unique(X[:, feature_index])
        if len(values) < 2:
            continue

        thresholds = (values[:-1] + values[1:]) / 2.0
        for threshold in thresholds:
            left_mask = X[:, feature_index] <= threshold
            right_mask = ~left_mask

            if not np.any(left_mask) or not np.any(right_mask):
                continue

            gini = self._split_gini(y[left_mask], y[right_mask])
            if gini < best_gini:

```



```
best_gini = gini
best_feature_index = feature_index
best_threshold = threshold

return best_feature_index, best_threshold
```

决策树的思想看起来比梯度下降直观得多，但真正写起来会发现，实现层面其实并不简单。递归建树、候选阈值搜索、停止条件控制，这些都要自己处理。这里用到的基尼指数形式是 $Gini(D) = 1 - \sum_k p_k^2$ ，如果一次划分之后左右子节点内部更“纯”，那么加权后的基尼值就会更小[8]。也正因为如此，在手动把一棵树写出来之后，我对 `DecisionTreeClassifier` 这种现成实现反而更能理解了，因为它替我们省掉的不只是几行代码，而是一大堆细碎但很重要的工作。

2.5 sklearn 对照实验设计

如果这次实验只写手搓版，其实也能说明“我知道算法怎么实现”。但我还是保留了 `sklearn` 对照，因为我不想只做成单纯的代码展示。手写版更像是在回答“我到底懂不懂这个模型”，`sklearn` 版则是在回答“真正做工程时，成熟实现和自己写的版本差多少”。把这两部分放在一起，我对这个实验的理解也会更完整。

后来我越来越觉得，这种对照特别有价值。因为很多时候我们以为自己“会了一个算法”，其实只是会调用它；但一旦把手写版和成熟库放在一起比较，就会发现，算法原理、数值稳定性、运行效率和工程可用性，本来就是几件不完全一样的事。

2.6 扩展模型选择依据

为了让实验不只停留在基础三模型，我又补了几类思路完全不同的方法。高斯朴素贝叶斯代表的是概率生成模型，它不是先去找一条边界，而是先假设不同类别的数据各自服从一定分布，再根据概率来判断类别。MLP 神经网络代表的是神经网络方法，它通过多层非线性变换去学习更复杂的特征关系。随机森林、AdaBoost 和 Gradient Boosting 则代表了集成学习，其中随机森林更偏向 bagging，后两者更偏向 boosting。

把这些方法放进同一个实验之后，比较的重点就不再只是“哪一个线性模型更快”，而变成了“不同类型的方法在同一个任务上到底会有什么差别”。这样一来，实验分析就不再局限于单一模型内部，而能够从概率建模、非线性建模和集成建模等不同角度观察模型差异。

至于概率图模型，这一类方法当然也很有代表性，比如 Bayesian Network 或者 HMM，在处理变量之间依赖关系比较明显、或者带有时序结构的数据时很有价值。但这次的 Iris 数据是一个比较小的静态表格数据集，特征之间没有特别明显的结构依赖，所以概率图模型并不是这次实验里最合适的重点。与其为了“看起来方法很多”硬加进去，不如把更适合当前任务的方法认真做好。所以这次我把概率建

模这一条主要放在了朴素贝叶斯上，而把概率图模型留在后续可以继续深入的方向里。

2.7 统一 Benchmark 计时方法

为了让比较结果尽量稳定，我给所有模型都加了统一的 **benchmark** 流程。核思路是先做一次预热，再重复多次训练和预测，最后取平均时间。这样可以减少首次调用和偶然波动对结果的影响。

```
def benchmark_model(model_factory, X_train, y_train, X_test, y_test, repeats=200):
    warmup_model = model_factory()
    warmup_model.fit(X_train, y_train)
    warmup_model.predict(X_test)

    total_fit_time = 0.0
    total_predict_time = 0.0
    final_predictions = None

    for _ in range(repeats):
        model = model_factory()

        fit_start = time.perf_counter()
        model.fit(X_train, y_train)
        total_fit_time += time.perf_counter() - fit_start

        predict_start = time.perf_counter()
        final_predictions = model.predict(X_test)
        total_predict_time += time.perf_counter() - predict_start

    return {
        "accuracy": float(accuracy_score_custom(y_test, final_predictions)),
        "fit_time_ms": (total_fit_time / repeats) * 1000,
        "predict_time_ms": (total_predict_time / repeats) * 1000,
    }
```

这种统一计时方式的意义，不只是多几个数字，而是让所有模型都在同一套标准下比较。否则如果每个模型测试方式都不一样，最后得出的“效率差异”其实也没有那么可信。

2.8 三分类任务扩展设计

为了让实验更有内容，我没有停留在作业原本要求的二分类 Iris 上，而是又把数据恢复成原始三分类任务，再观察不同模型在更复杂条件下的表现。这里最重要的一点，是我没有直接放弃手写模型，而是给手写逻辑回归和手写线性 SVM 又加了一层自己写的 **One-vs-Rest** 封装，让它们也能够参与三分类比较。


```

class OneVsRestScratchClassifier:
    def __init__(self, base_model_factory):
        self.base_model_factory = base_model_factory
        self.classes_ = None
        self.models = []

    def fit(self, X, y):
        self.classes_ = np.unique(y)
        self.models = []

        for class_label in self.classes_:
            model = self.base_model_factory()
            binary_targets = (y == class_label).astype(int)
            model.fit(X, binary_targets)
            self.models.append(model)

    def predict(self, X):
        score_matrix = np.column_stack(
            [self._get_class_scores(model, X) for model in self.models]
        )
        best_indices = np.argmax(score_matrix, axis=1)
        return self.classes_[best_indices]

```

这一步的意义在于，它并不是单纯增加几个现成模型，而是把原本只支持二分类的手写模型继续扩展到更接近原始任务的条件下参与比较。这样一来，手写模型不再只服务于基础二分类实验，而是能够进入更完整的多分类对照分析。

2.9 相关研究与扩展实验依据

从近几年的相关研究看，表格数据分类并不是“深度网络一定优于传统模型”的简单场景。Grinsztajn 等人的工作指出，树模型在大量表格任务上仍然具有明显优势，这种优势与数据的非平滑性、样本规模和无关特征等因素密切相关[1]。Gorishniy 等人在统一实验协议下重新比较了多种表格深度学习模型，结论同样认为深度学习与梯度提升树之间并不存在普适性的单方面优势[2]。另一方面，TabTransformer 等工作也表明，当特征交互更复杂时，注意力机制在表格任务中依然具有进一步扩展的潜力[3]。

除了精度之外，概率输出本身的可靠性也是分类任务里一个常被忽略但非常重要的问题。Guo 等人的研究表明，现代神经网络即便在准确率不低的情况下，也可能给出明显失准的预测概率[4]。因此，仅比较最终准确率并不足以完整评价一个模型的实际可用性。

基于这些研究结论，本实验在基础准确率、训练时间和三分类稳定性之外，又增加了样本效率实验、概率校准实验和温度缩放分析。这样设计之后，我不仅能观察模型“分得是否正确”，还能够继续比较它们“在小样本条件下能否稳定学习”以及“预测概率是否能够反映真实置信水平”，使整个实验更接近研究型比较的展开方式。

2.10 扩展模型算法原理补充

为了让扩展实验不只是“多跑几个模型”，我也把这些方法背后的核心思路重新梳理了一遍。高斯朴素贝叶斯属于概率生成模型，它不是直接学习分类边界，而是先估计每个类别的先验概率和条件分布，然后用贝叶斯公式计算后验概率。它最关键的假设是特征在给定类别条件下相互独立，因此后验概率可以写成 $P(y|x) \propto P(y) \prod_j P(x_j|y)$ [5][6]。这个假设在很多真实任务里并不严格成立，但它换来的是非常轻量的训练和推断过程。

随机森林、AdaBoost 和 Gradient Boosting 都属于集成学习，但三者的出发点并不相同。随机森林更接近 bagging 思路，它通过 bootstrap 采样和随机特征子集训练多棵树，再通过投票得到最终结果，因此更强调方差降低和整体稳定性[9]。AdaBoost 则会不断提高先前分错样本的权重，让后续弱分类器把注意力集中到难样本上，最终得到一个加权组合分类器[10]。Gradient Boosting 的思路更进一步，它把迭代训练看成在函数空间里做逐步优化，每一轮都让新树去逼近当前损失函数的负梯度，因此更像是在不断修正已有模型留下的误差[11]。

多层感知机 MLP 代表的是最基础的前馈神经网络。它通过多层线性变换和非线性激活不断构造新的特征表示，可以写成 $h^{(l+1)} = \phi(W^{(l)}h^{(l)} + b^{(l)})$ 。和线性模型相比，MLP 的优势在于能够拟合更复杂的非线性关系，但代价是参数更多、训练时间更长，对数据规模和超参数也更敏感[12]。也正因为这些差异，我才在后面的实验里专门把线性模型、树模型、集成模型和神经网络放到一起比较。

2.11 可视化分析设计

如果只用表格列出准确率和时间，很多模型之间的差异其实还是比较抽象。所以在这次扩展里，我又补了几类更偏“结构理解”的可视化。数据分布图主要用来观察不同类别在特征空间里的天然分离程度；决策区域图用来直接看边界形状，判断线性模型和树模型在划分方式上的差异；ROC 曲线可以比较二分类模型在不同阈值下的整体判别能力；线性模型系数图能够展示每个特征对分类方向的贡献；集成模型特征重要性图则更适合观察树模型在分裂过程中到底更依赖哪些特征。

我之所以补这些图，是因为它们分别对应了几种不同层次的问题。准确率是在问“最后分对了多少”，决策区域是在问“它是怎么分的”，系数和特征重要性是在问“它更依赖什么信息”，而 ROC 和概率校准则是在问“它的分数和概率是否可靠”。把这些角度合在一起之后，我对模型行为的理解会比只看单一结果更完整。

3. 实验结果与对比分析

3.1 核心算法手写实现与 sklearn 对照

首先，我对逻辑回归、线性 SVM 和决策树分别做了“手写版 vs sklearn 版”的对照实验。每个模型重复运行 200 次，取平均训练时间和平均预测时间，结果如下：

表 3-1 核心算法手写实现与 sklearn 对照结果

算法	实现方式	准确率	平均训练时间(ms)	平均预测时间(ms)
Logistic Regression	Scratch	1.0000	54.7410	0.0142
Logistic Regression	sklearn	1.0000	1.0310	0.1051
Linear SVM	Scratch	1.0000	67.8956	0.0085
Linear SVM	sklearn	1.0000	0.5158	0.1811
Decision Tree	Scratch	1.0000	2.2567	0.0135
Decision Tree	sklearn	1.0000	0.5642	0.1166

从准确率上看，这三种算法在当前这份二分类 Iris 数据集上都达到了 **1.0000**。这说明三种手搓版模型的基本逻辑是成立的，它们确实学到了能够完成分类的边界或规则，而不是只是“写出了一个看起来像算法的程序”。

如果把注意力转到训练效率上，差异就会立刻变得明显。逻辑回归训练时，**sklearn** 大约快了 **53.09** 倍；线性 SVM 大约快了 **131.64** 倍；决策树也快了 **4.00** 倍。这个结果非常符合直觉，因为成熟库的底层实现已经做了大量性能优化，而 **Python** 手写版本即便逻辑正确，也不可避免地会在循环、递归和数值计算上付出更多解释器开销。需要说明的是，毫秒级时间测试会随着机器当前状态略有波动，但整体趋势是非常稳定的。

下面这张图展示了三种核心算法在训练阶段的时间对比。因为手写版和 **sklearn** 版之间差距比较大，纵轴使用了对数刻度，这样更容易看清量级差别。



图 3-1 三种核心算法训练时间对比图

从图上可以很直观地看出，训练效率上的优势基本都在成熟库这边，尤其在线性 SVM 上表现得最明显。这说明算法原理的正确实现与工程层面的高效实现之间仍然存在明显差别，成熟库在数值优化和底层实现上的优势是十分明显的。

预测时间的结果则有一点反直觉。在这次实验里，手写版的平均预测时间反而更低一些。

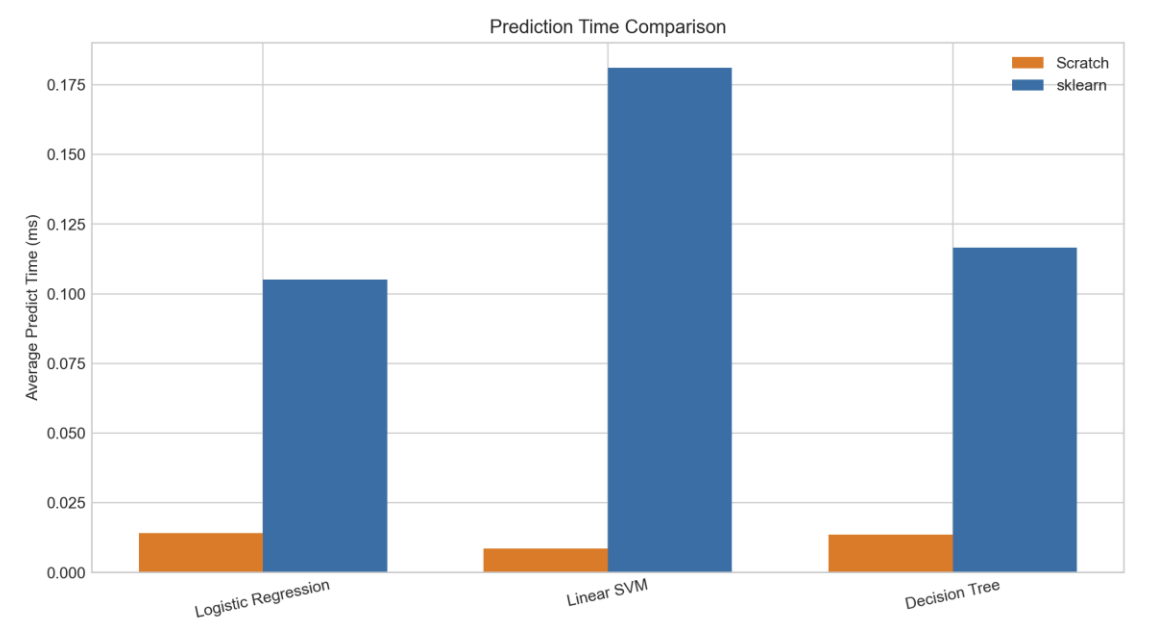


图 3-2 三种核心算法预测时间对比图

但我并不认为应该直接据此下结论说“手写版预测更快”。更合理的解释是，这次测试集规模非常小，单次调用的固定开销在总时间里占比过高，所以这里的预测时间更像是在反映函数封装和对象接口的开销，而不是模型在大规模数据上的真实吞吐能力。因此在这个实验中，训练时间的比较其实更有代表性。

3.2 扩展模型比较分析

如果实验只做到这里，其实已经能完成基础要求了。但为了让比较更完整，我继续补上了高斯朴素贝叶斯、MLP 神经网络、随机森林、AdaBoost 和 Gradient Boosting。这一步做完之后，实验就不再局限于三种经典模型，而是开始覆盖概率生成、神经网络和集成学习这些完全不同的建模思路。每个扩展模型重复运行 50 次，结果如下：

表 3-2 扩展模型比较结果

方法	所属方法族	准确率	平均训练时间 (ms)	平均预测时间 (ms)
Gaussian Naive Bayes	概率生成模型	1.0000	0.3891	0.1152
MLP Neural Network	神经网络	1.0000	117.8627	0.2116
Random Forest	集成学习 Bagging	1.0000	94.4089	4.3390

AdaBoost	集成学习 Boosting	1.0000	1.7488	0.2831
Gradient Boosting	集成学习 Boosting	1.0000	38.6923	0.3388

单看准确率，这些方法在当前任务上依然全部达到了 **1.0000**。这恰恰说明了一个很有意思的事实：对这样一个规模不大、类别区分相对清晰的二分类 Iris 任务来说，很多不同方法都能够顺利完成分类，准确率并不是最能拉开差距的指标。真正拉开差距的，反而是训练成本、预测成本、模型复杂度以及它们背后所代表的建模思路。

下面这张图展示了扩展方法在训练时间上的差异。

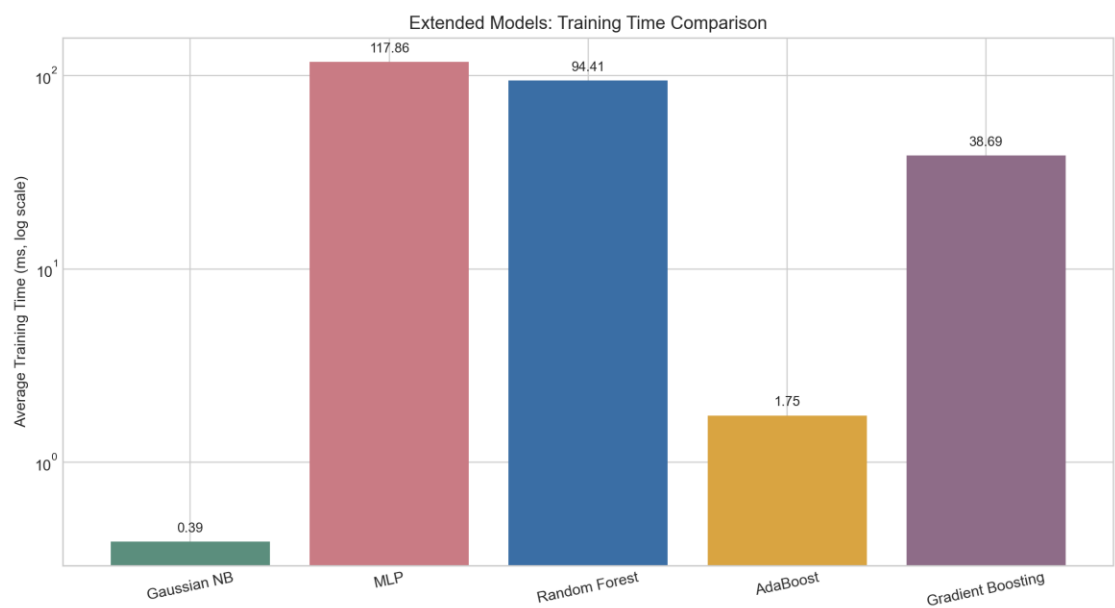


图 3-3 扩展模型训练时间对比图

从图中可以看到，高斯朴素贝叶斯训练速度最快，几乎只需要极短的时间就可以完成拟合；MLP 神经网络是这组方法里训练最慢的，这和它需要进行多轮非线性参数更新是吻合的；随机森林和梯度提升树居中偏慢，而 AdaBoost 在这个任务上表现得相当轻量。也就是说，当准确率已经都差不多时，训练速度和模型结构的复杂程度就开始变得非常重要。

预测阶段的差异如下图所示。

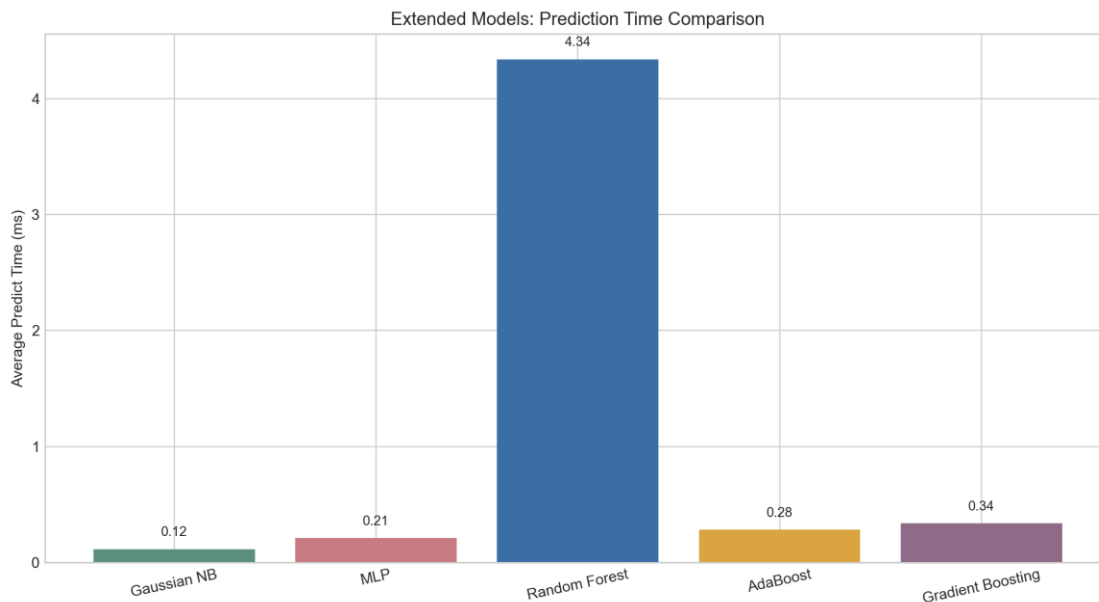


图 3-4 扩展模型预测时间对比图

在这组方法里，高斯朴素贝叶斯依然是预测最快的，而随机森林的预测耗时明显更高。这也很符合模型结构本身的特点，因为随机森林需要在多棵树上完成投票，而朴素贝叶斯的推断过程则要简洁得多。虽然这次数据规模不大，但这种差异已经能够让人直观感受到，不同模型“准确率一样”并不意味着它们在实际应用里没有区别。

3.3 原始三分类任务扩展实验

前面的实验主要建立在二分类 Iris 上，这样做的好处是方便理解和展示，但难度相对有限。为了让实验更有层次，我又把任务扩展回原始三分类 Iris，并选了 6 个具有代表性的模型继续比较，包括两种扩展后的手写模型、手写决策树、朴素贝叶斯、随机森林和 MLP 神经网络。为了避免偶然划分带来的影响，这一部分不再只看单次结果，而是在 20 个不同随机种子下重复划分训练集和测试集，再统计平均准确率和标准差。

表 3-3 原始三分类任务稳定性结果

模型	方法类型	平均准确率	标准差
Scratch OvR Logistic	手写线性模型	0.9283	0.0338
Scratch OvR SVM	手写线性模型	0.6933	0.0854
Scratch Decision Tree	手写树模型	0.9333	0.0333
Gaussian Naive Bayes	概率模型	0.9483	0.0307
Random Forest	集成学习	0.9467	0.0306
MLP Neural Network	神经网络	0.9350	0.0387

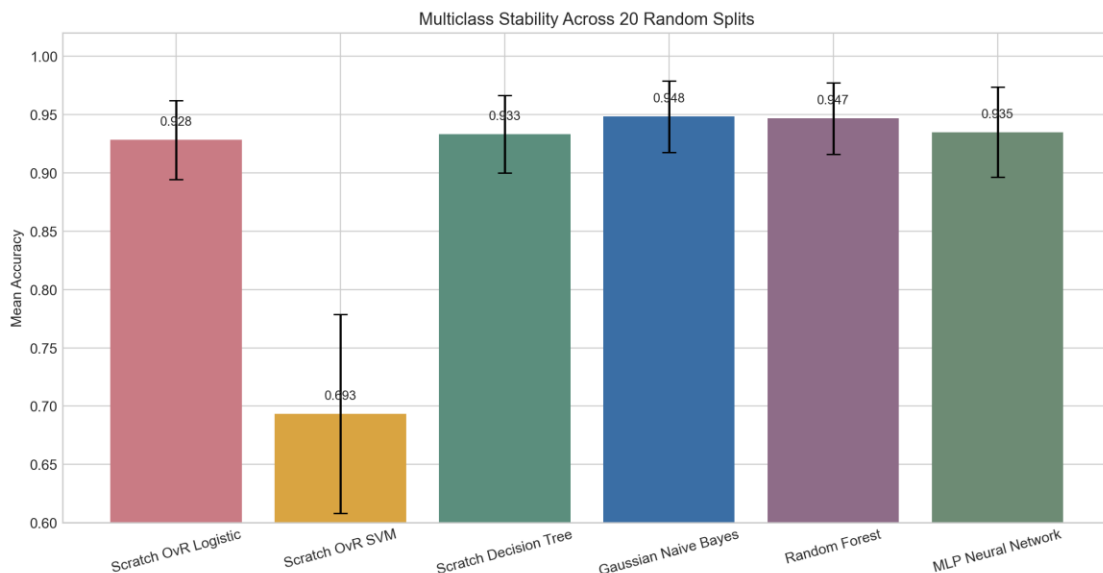


图 3-5 原始三分类任务下的稳定性对比图

这部分结果一下子让实验变得更有比较价值了。因为在三分类任务下，模型之间终于不再全部都是满分。可以看到，高斯朴素贝叶斯和随机森林的平均准确率最高，都稳定在 **0.94** 以上；手写 One-vs-Rest 逻辑回归也达到了 **0.9283**，说明这种扩展方式是有效的；手写决策树表现也比较稳定。比较特别的是手写 One-vs-Rest SVM，它在这个任务上的平均准确率只有 **0.6933**，波动也明显更大。这说明同样是把二分类模型扩成三分类，不同基础模型对 OvR 策略的适配程度并不一样，这一点比单纯看二分类结果更能说明问题。

这一节的价值并不只在于比较不同模型的准确率高低，更在于它把实验从简化后的二分类任务进一步扩展回原始三分类问题，从而使比较结果更接近数据集本身的真实难度，也使模型之间的差异更容易被观察到。

3.4 噪声扰动鲁棒性分析

除了看正常数据下的准确率，我还想知道一个更实际的问题：如果测试数据本身带有噪声，不同模型的表现会掉得有多快。为了做这个分析，我在原始三分类 Iris 的测试集上逐步加入不同强度的高斯噪声，噪声标准差从 **0.0** 增加到 **1.0**，并对每个噪声等级重复 **20** 次取平均准确率。

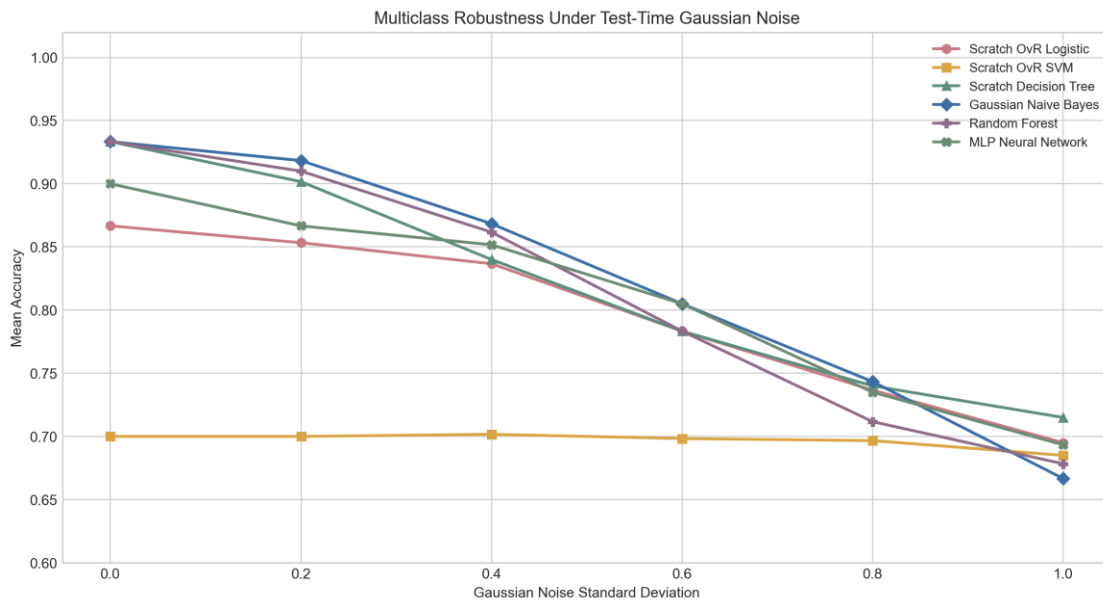


图 3-6 原始三分类任务下的抗噪声能力对比图

从图中可以看到，随着噪声逐渐变大，几乎所有模型的准确率都在下降，这一点很符合直觉，因为输入特征被扰动之后，类别边界自然会变得更难判断。不过不同模型下降的速度并不完全一样。当噪声标准差增加到 **1.0** 时，手写决策树还能保持 **0.7150** 的平均准确率，是这组模型里表现最稳的；手写逻辑回归和 MLP 神经网络分别保持在 **0.6950** 和 **0.6933** 左右；朴素贝叶斯下降到 **0.6667**，相对更容易受到噪声影响。

这一节的意义在于，它把实验从“谁在干净数据上表现更好”进一步扩展为“谁在数据受扰动时更稳定”。在实际应用中，输入数据往往不可避免地带有噪声，因此鲁棒性分析能够补充单纯准确率所无法反映的信息。

3.5 特征消融实验分析

为了继续往下挖，我又做了特征消融实验。具体做法是每次去掉一个特征，然后重新在原始三分类任务上训练和测试，最后看平均准确率会下降多少。这个实验能帮助判断，模型到底更依赖哪些特征。

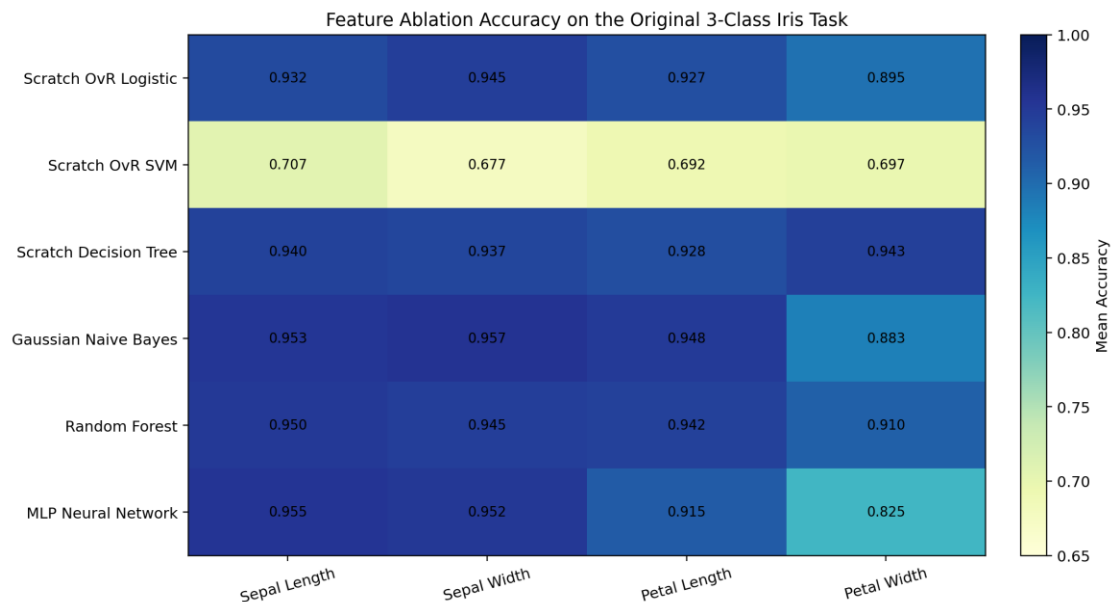


图 3-7 原始三分类任务下的特征消融热力图

从热力图可以看出，不同模型对特征的依赖方式并不完全一样，但整体上还是能看到一些比较一致的现象。对高斯朴素贝叶斯、随机森林和 MLP 神经网络来说，去掉 **Petal Width** 之后准确率下降得比较明显，尤其是 MLP，从 **0.9350** 左右下降到了 **0.8250**，说明这个特征对它非常关键。手写逻辑回归在去掉 **Petal Width** 后也降到了 **0.8950**。相比之下，去掉 **Sepal Width** 对大多数模型影响都相对小一些。

这部分分析的作用在于，它把讨论重点从“模型结果是否足够好”推进到“模型到底依赖了哪些信息”。通过特征消融，可以更具象地观察不同模型对输入特征的敏感性，从而更深入地理解模型与数据之间的关系。

3.6 混淆矩阵结果分析

为了看得更具体一些，我还选了几个代表性模型，在固定随机划分下画出了混淆矩阵。

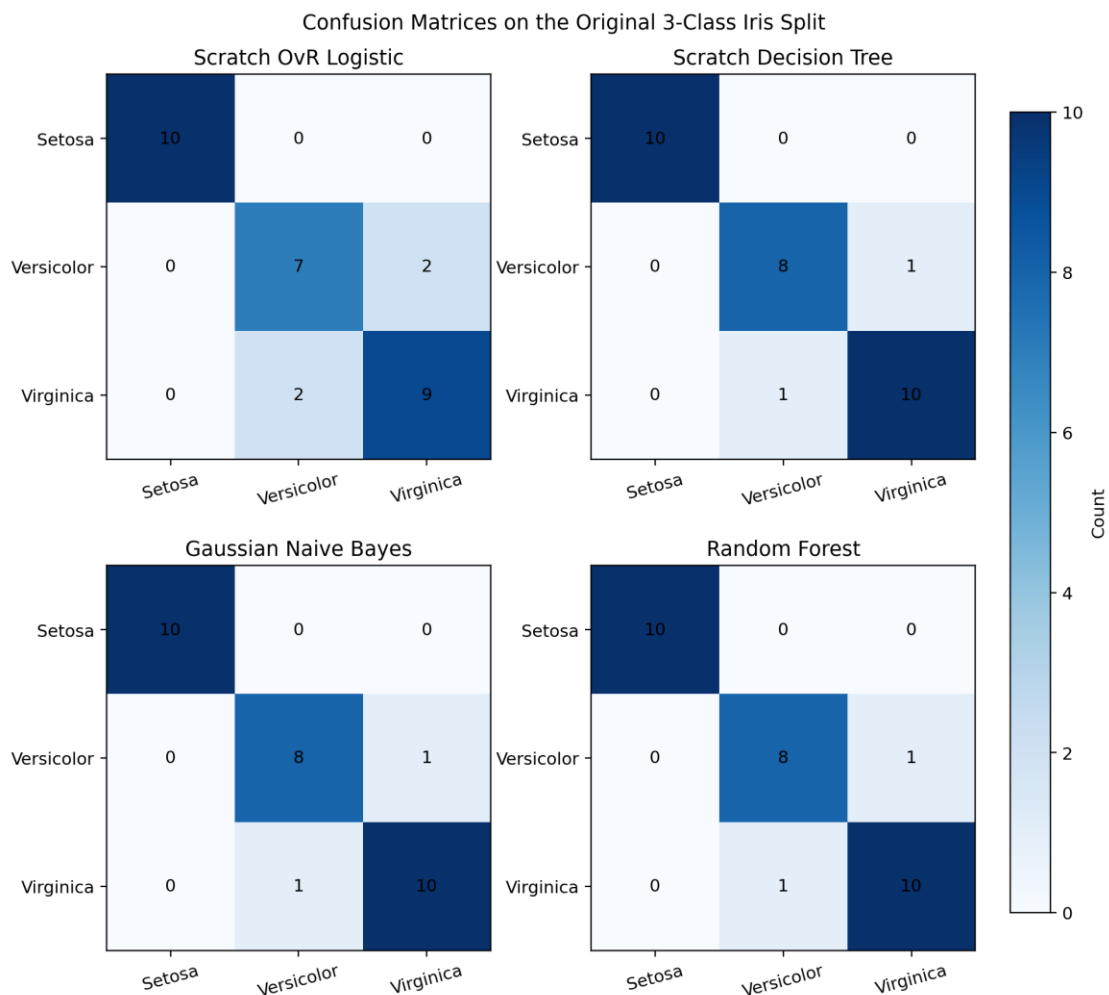


图 3-8 原始三分类任务下的混淆矩阵对比图

这部分结果很有代表性。几乎所有模型都能把 **Setosa** 分得非常准确，基本没有出现误分；真正容易混淆的还是 **Versicolor** 和 **Virginica** 两类。比如手写 OvR 逻辑回归在这个划分下把 **Versicolor** 和 **Virginica** 各错分了 2 个，而随机森林、高斯朴素贝叶斯和手写决策树都只各错了 1 个。这说明在 **Iris** 数据里，第一类本身就比较容易区分，而后两类之间的边界才是模型真正需要下功夫的地方。

混淆矩阵的作用在于，它能够把“错误集中出现在哪里”展示得更具体。相比单一准确率指标，它更适合用于观察模型究竟是整体表现不稳定，还是主要在少数相近类别之间发生混淆。

3.7 扩展实验综合分析

通过引入概率生成模型、神经网络、集成学习，以及三分类扩展、噪声鲁棒性和特征消融等补充分析，实验评价维度从单一准确率进一步扩展到了训练效率、稳定性、鲁棒性和特征依赖性。这样一来，不同模型之间的差异就不再只体现在“最终分数”上，而能够从多个角度得到更系统的比较。

这种多维度分析方式的意义在于，它能够更完整地反映模型在不同条件下的行为特征。对于表格分类任务而言，仅比较一次训练测试结果往往不足以支撑全面结论，而结合多分类任务、扰动测试和特征消融之后，实验结论会更加充分，也更具分析价值。

3.8 深度学习实现方式对比实验

在前面的实验中，MLP 只作为一种补充模型出现。为了进一步丰富实验内容，我又单独设计了一组深度学习实现方式对比实验，重点比较不同框架、网络深度、正则化策略和优化器设置对结果的影响。具体来说，这一部分一共比较了 6 种实现方式，包括 sklearn 的浅层与深层 MLP、PyTorch 的浅层和深层 MLP，以及带有 BatchNorm + Dropout 和 SGD 优化器的两种变体。所有模型都在原始三分类 Iris 数据上进行 10 个随机划分实验，并统计平均准确率、标准差、平均训练时间和参数量。

表 3-4 深度学习实现方式对比结果

模型	框架	结构说明	平均准确率	标准差	平均训练时间(ms)	参数量
sklearn Shallow MLP	sklearn	1 hidden layer (8), ReLU, Adam	0.9500	0.0269	155.90	67
sklearn Deep MLP	sklearn	3 hidden layers (32,16,8), ReLU, Adam	0.9567	0.0300	199.81	851
Torch Shallow MLP	PyTorch	1 hidden layer (16), ReLU, Adam	0.9467	0.0371	1455.12	131
Torch Deep MLP	PyTorch	3 hidden layers (32,16,8), ReLU, Adam	0.9400	0.0512	1851.56	851
Torch BN+Dropout MLP	PyTorch	3 hidden layers + batchnorm + dropout	0.9300	0.0605	2652.45	3203
Torch SGD MLP	PyTorch	3 hidden layers (32,16,8), ReLU, SGD	0.9433	0.0473	1139.55	851

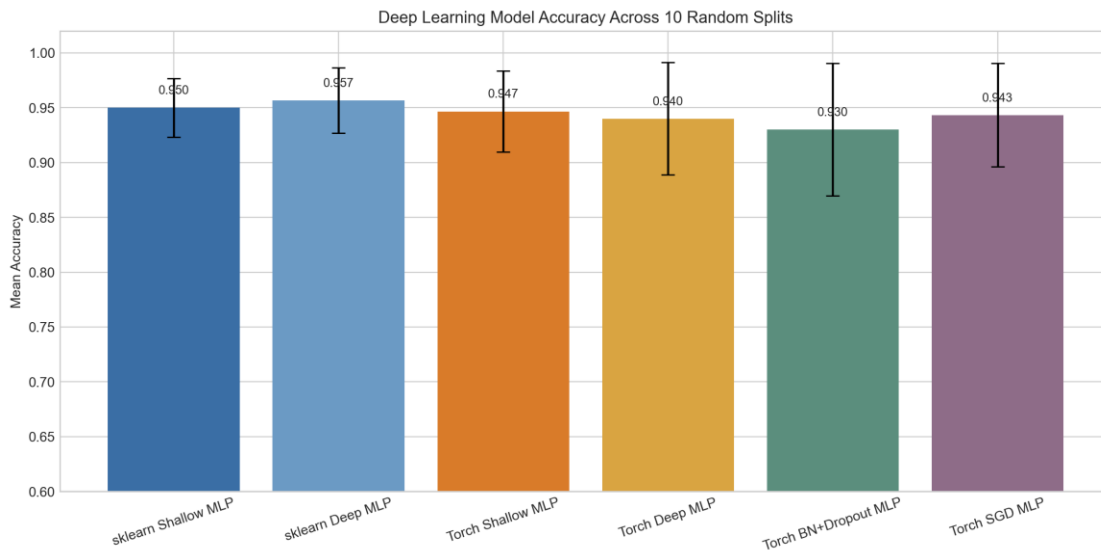


图 3-9 深度学习实现方式准确率对比图

从整体结果来看，`sklearn Deep MLP` 的平均准确率最高，达到 **0.9567**，同时波动也控制在较小范围内；`sklearn Shallow MLP` 紧随其后，为 **0.9500**。相比之下，`PyTorch` 实现虽然提供了更灵活的结构控制能力，但在当前这个小样本结构化数据任务上，并没有因为网络更深或者正则化更强而明显优于 `sklearn` 版本。特别是 `Torch BN+Dropout MLP`，虽然结构最复杂、参数量最大，但平均准确率只有 **0.9300**，训练时间也最长。这说明在 `Iris` 这种规模较小、特征维度较低的数据集上，更复杂的深度学习结构并不一定带来更好的效果。

3.9 深度学习模型训练过程分析

除了比较最终准确率之外，我还进一步观察了这些深度学习模型在固定随机划分下的训练损失变化过程。这个分析能够帮助判断不同实现方式的收敛速度与训练稳定性，而不仅仅是比较最终结果。

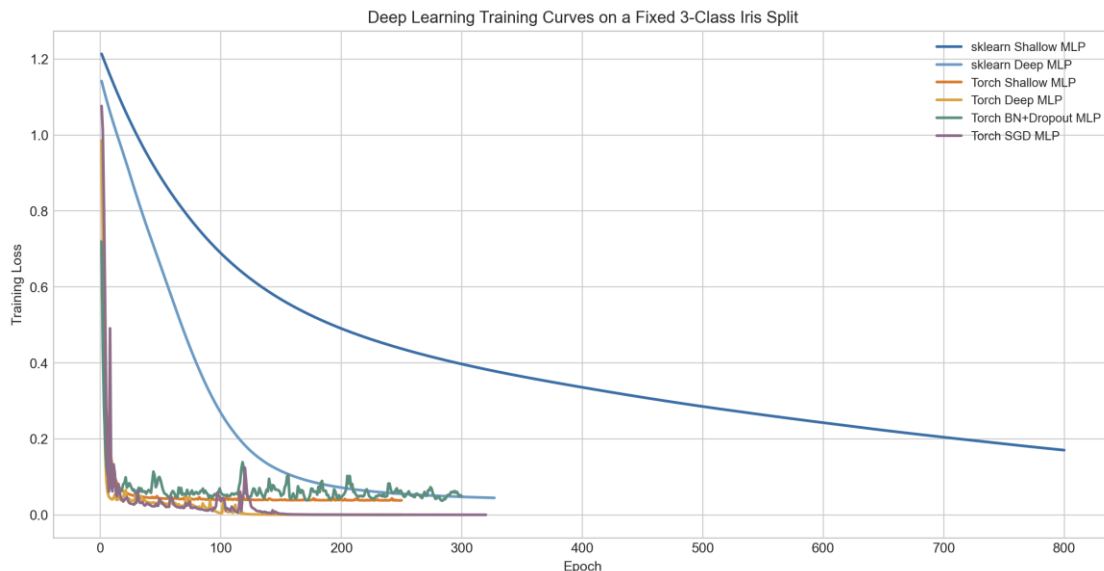


图 3-10 深度学习模型训练曲线对比图

从训练曲线可以看出，不同模型的收敛特征有比较明显的区别。`sklearn` 的两种 MLP 曲线整体更平滑，损失下降过程也更稳定；`PyTorch` 版本在前期下降较快，但不同结构之间波动差别较大。特别是带 `BatchNorm + Dropout` 的模型，训练曲线相对起伏更明显，这与正则化引入的扰动有关。`Torch SGD MLP` 在训练后期仍然能够继续下降，但达到稳定状态所需的迭代轮数更多。这个现象说明，不同深度学习实现方式之间的差异不仅体现在最终精度上，也体现在训练过程本身的稳定性与收敛效率上。

3.10 深度学习模型鲁棒性与复杂度分析

在深度学习实现方式对比实验中，我还进一步加入了噪声扰动测试，用于观察模型在测试数据被高斯噪声污染时的表现变化。下表给出了噪声标准差为 **1.0** 时各模型的平均准确率，以及相对于无噪声条件下的下降幅度。

表 3-5 深度学习模型高噪声条件结果

模型	无噪声准确率	噪声标准差为 1.0 时准确率	准确率下降幅度
sklearn Shallow MLP	0.9000	0.6617	0.2383
sklearn Deep MLP	0.9000	0.6950	0.2050
Torch Shallow MLP	1.0000	0.6817	0.3183
Torch Deep MLP	1.0000	0.6717	0.3283
Torch BN+Dropout MLP	0.9333	0.6300	0.3033
Torch SGD MLP	0.9667	0.6650	0.3017

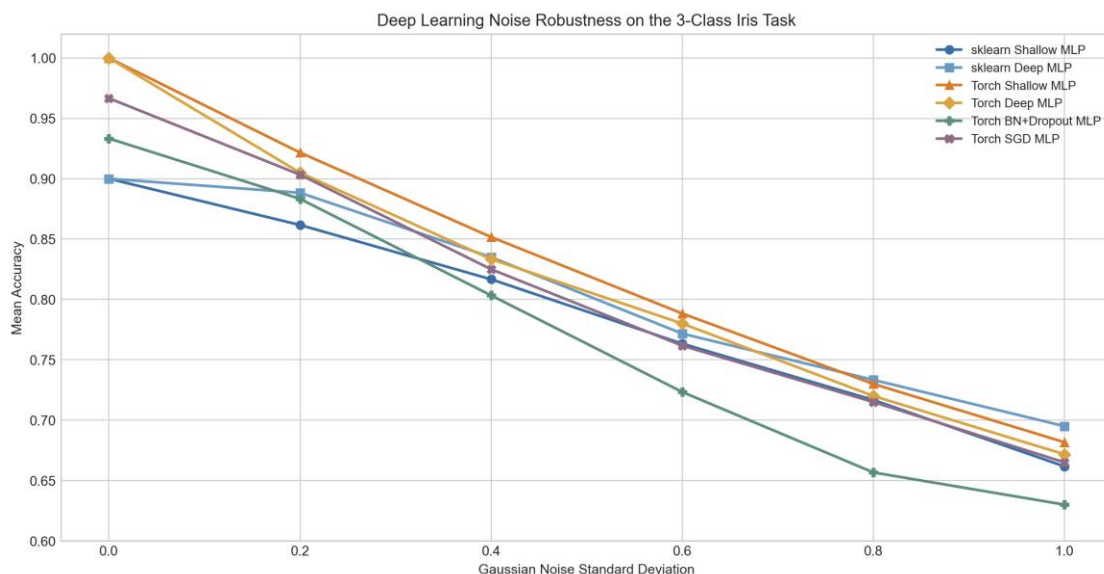


图 3-11 深度学习模型抗噪声能力对比图

从这组结果来看，**sklearn Deep MLP** 在高噪声条件下的保持能力最好，准确率下降幅度约为 **0.2050**；**sklearn Shallow MLP** 也保持了相对稳定的鲁棒性。相比之下，**PyTorch** 深层模型在当前设置下对噪声更敏感，尤其是深层 **Adam** 版本和浅层 **PyTorch** 版本，下降幅度都超过了 **0.31**。这说明在小样本数据集上，深度学习模型的复杂化并不必然带来更强的鲁棒性，反而可能因为模型表达能力过强而对输入扰动更加敏感。

为了进一步比较模型复杂度与性能之间的关系，我还把参数量和平均准确率放到同一张图里进行展示，点的大小表示平均训练时间。

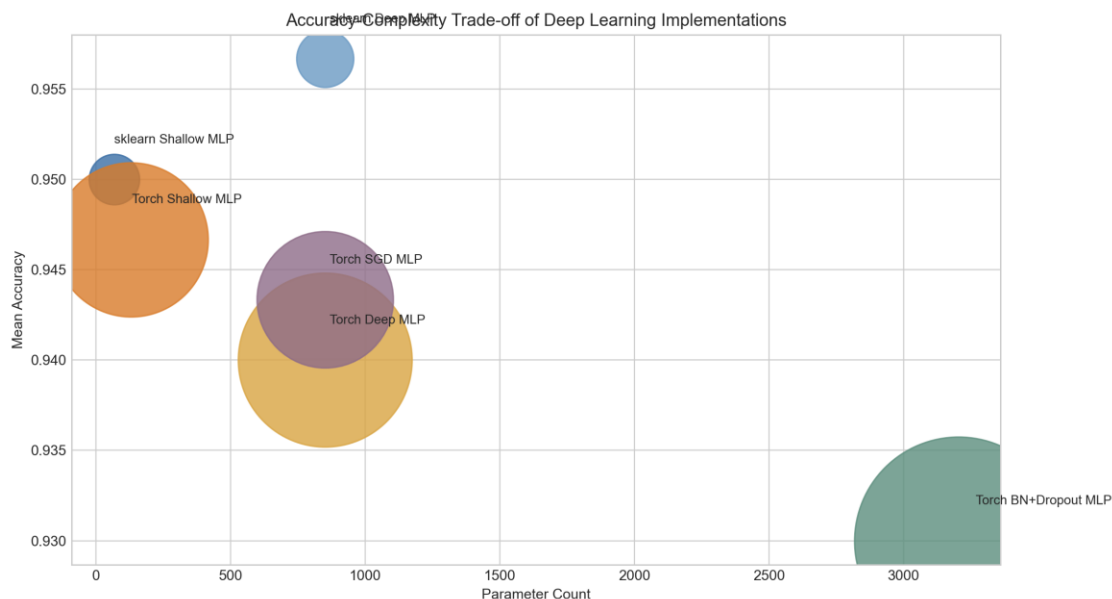


图 3-12 深度学习模型参数量与性能权衡图

从参数量与性能的关系可以看到，参数更多并不意味着一定更好。`sklearn` Deep MLP 和 Torch Deep MLP 虽然参数量相同，都是 851，但前者在准确率和训练效率上都更优；Torch BN+Dropout MLP 的参数量达到 3203，远高于其他模型，但准确率并没有相应提升。这个结果进一步说明，在当前数据规模下，模型复杂度与最终效果之间并不是简单的正相关关系，合理的结构设计比单纯增加层数和参数量更重要。

3.11 样本效率与训练规模分析

为了进一步观察不同方法对训练样本数量的敏感程度，我又补充了一组样本效率实验。具体做法是固定测试集划分，只把训练集按 20%、40%、60%、80% 和 100% 五个比例逐步展开，然后分别比较高斯朴素贝叶斯、随机森林、`sklearn` Deep MLP 和 Torch Deep MLP 的平均准确率。这样的设计可以更直接地回答一个问题：当可用样本并不充足时，哪一类方法更容易先学到稳定的决策边界。

表 3-6 样本效率实验结果

模型	20% 训练集	40% 训练集	60% 训练集	80% 训练集	100% 训练集
Gaussian Naive Bayes	0.9400	0.9400	0.9533	0.9567	0.9500
Random Forest	0.9367	0.9233	0.9367	0.9500	0.9467
sklearn Deep MLP	0.9500	0.9267	0.9267	0.9467	0.9567
Torch Deep MLP	0.9333	0.9167	0.9133	0.9367	0.9367

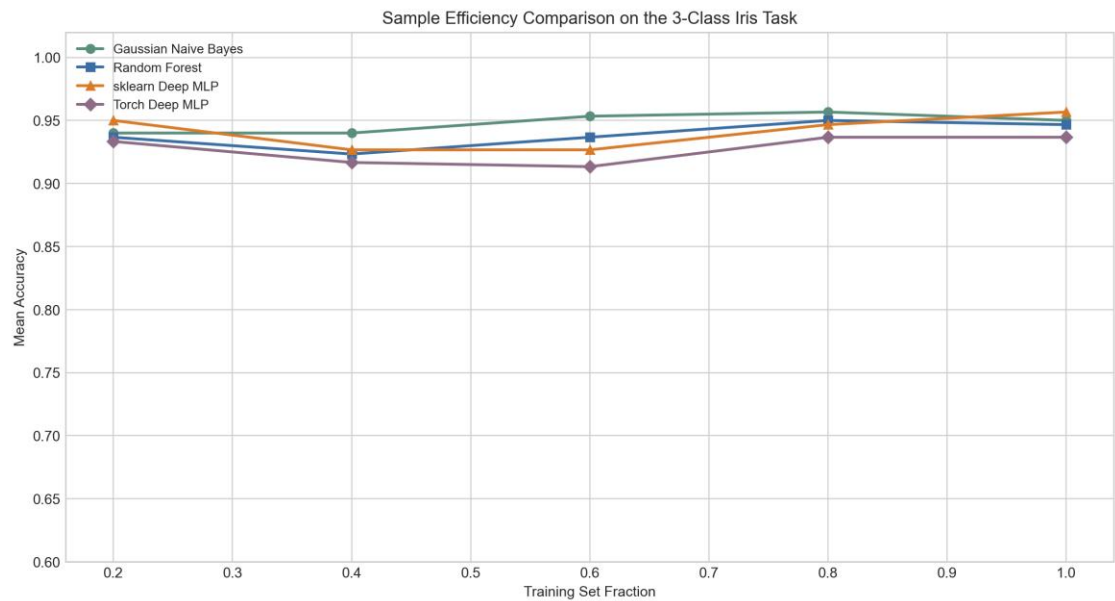


图 3-13 深度学习与传统模型样本效率对比图

从结果可以看出，传统表格模型在小样本条件下的表现更稳。高斯朴素贝叶斯在只有 20% 训练数据时就已经达到 0.9400，之后整体波动很小；随机森林虽然在 40% 比例处略有回落，但总体也保持在 0.92 以上。相比之下，深度模型对训练样本数量更加敏感。sklearn Deep MLP 在完整训练集上可以达到最高的 0.9567，但在中等样本比例时并没有稳定压过传统模型；Torch Deep MLP 则在整个过程中都略低于 sklearn 版本。

这一组结果和前述相关研究的结论比较一致。对于 Iris 这类规模较小、结构化特征比较明确的表格任务，传统方法在样本效率上通常更有优势，而深度学习模型要想真正体现结构优势，往往需要更大的样本规模或者更复杂的特征交互场景。这也说明，在表格数据任务上讨论模型优劣，不能只看完整数据下的一次最好结果，还需要考虑模型随着训练规模变化的收益速度。

3.12 概率校准与可靠性分析

在很多实际场景里，分类器不仅要输出标签，还要输出可信的概率。例如在医学筛查、风险预警或者后续阈值决策中，0.95 的预测概率和 0.65 的预测概率代表的是完全不同的含义。因此，在准确率之外，我又增加了一组概率校准实验，比较模型预测置信度与真实正确率之间的一致程度。这里使用了 ECE、Brier Score 和 NLL 三个指标，并额外对 Torch Deep MLP 做了温度缩放处理。

表 3-7 概率校准实验结果

模型	准确率	ECE	Brier Score	NLL	平均置信度
Gaussian Naive Bayes	0.9333	0.0414	0.1024	0.1463	0.9747
Random Forest	0.9333	0.0405	0.0790	0.1166	0.9451
sklearn Deep MLP	0.9000	0.0793	0.1419	0.2305	0.9702
Torch Deep MLP	0.9000	0.0855	0.1550	0.5945	0.9855
Torch Deep MLP + Temp Scaling	0.9000	0.0523	0.1234	0.2302	0.9007

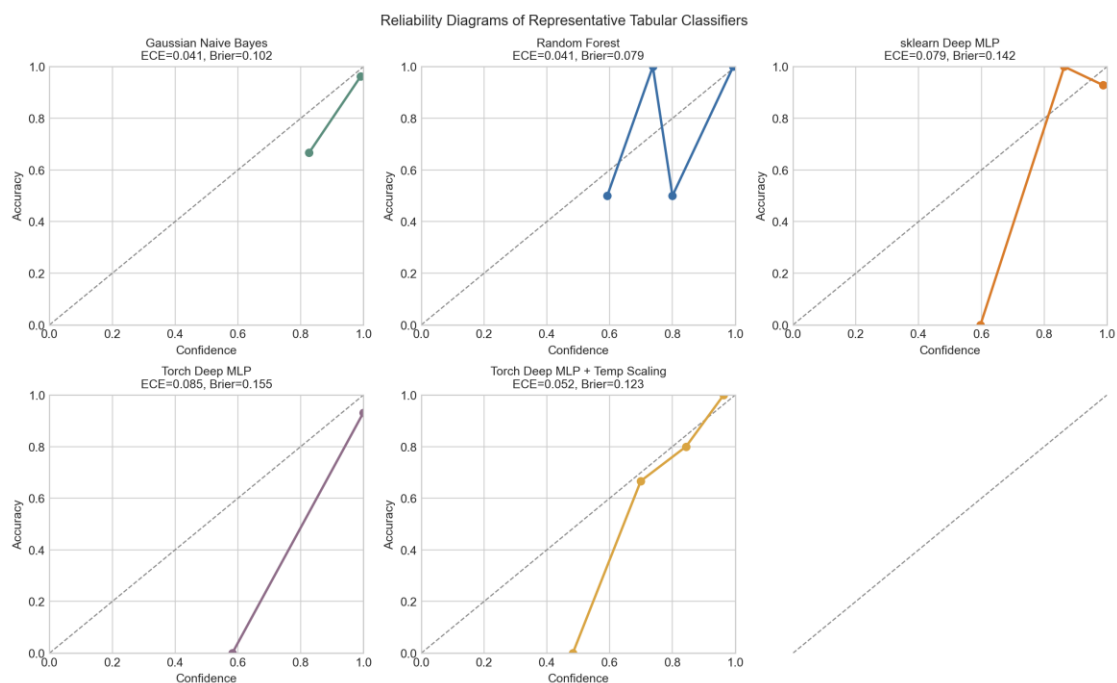


图 3-14 代表性模型可靠性图

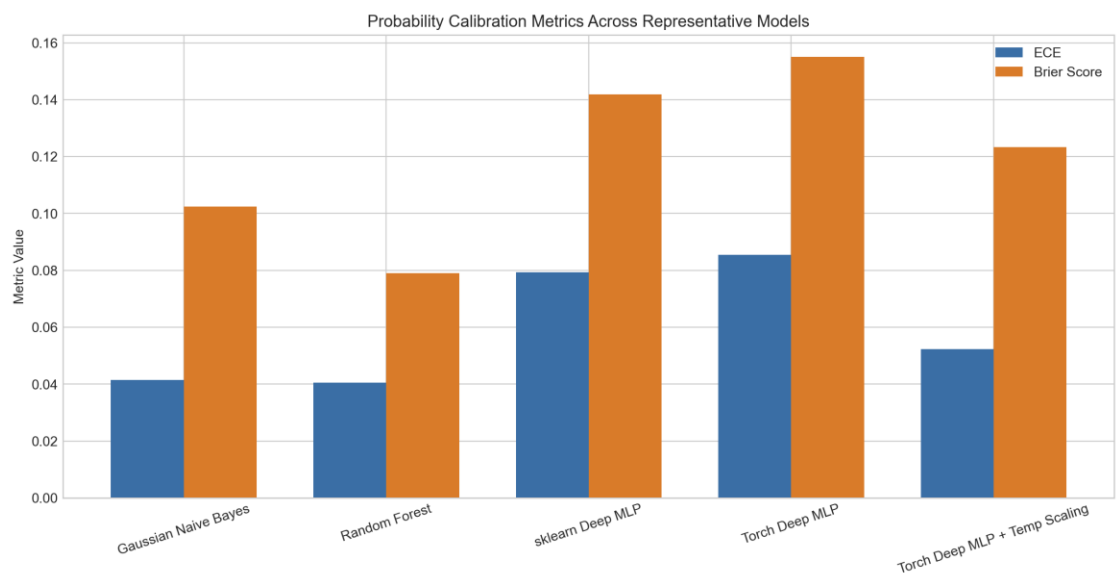


图 3-15 概率校准指标对比图

从这部分结果里，我看到了一个很有意思的现象。**Random Forest** 和 **Gaussian Naive Bayes** 在当前任务上不仅准确率较高，而且 **ECE** 也更低，说明它们给出的概率输出与真正准确率更加接近。相反，两个深度 **MLP** 虽然准确率并不算低，但平均置信度都接近甚至超过 **0.97**，而实际准确率只有 **0.90**，表现出比较明显的过度自信。其中 **Torch Deep MLP** 的 **ECE** 达到 **0.0855**，**NLL** 也明显高于其他模型，这说明它的概率输出存在较强的失准现象。

进一步做温度缩放之后，Torch Deep MLP 的分类准确率保持不变，仍然是 0.9000，但 ECE 从 0.0855 降到了 0.0523，NLL 也从 0.5945 明显下降到 0.2302。这说明温度缩放虽然不会直接改变分类边界，却能够有效缓和神经网络的概率过度自信问题。换句话说，如果只看准确率，这个改进几乎是不可见的；但如果把研究视角扩展到概率质量，就能看到模型行为已经发生了很明显的改善。这一现象也和相关校准研究中对于现代神经网络“高准确率并不意味着高可靠性”的结论是一致的。

3.13 数据分布与决策区域可视化

除了前面的结果表和统计图之外，我还补了几张更直观的结构图，用来直接观察数据分布和模型边界。先看 Iris 数据本身在特征空间里的分布情况。

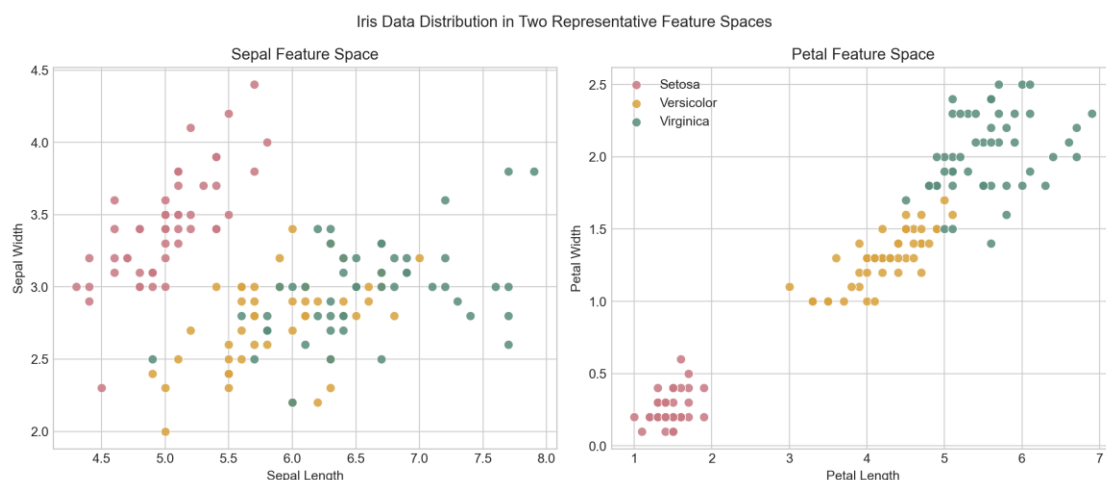


图 3-16 Iris 数据在典型特征空间中的分布图

从这张图里可以很直观地看出，Sepal Length 和 Sepal Width 两个萼片特征虽然也能提供一定区分信息，但三类样本之间仍然有明显重叠；相比之下，Petal Length 和 Petal Width 形成的花瓣特征空间分离程度要清楚得多，Setosa 基本可以单独分离出来，Versicolor 与 Virginica 则主要在边界附近发生交叠。也就是说，在模型训练之前，数据本身就已经在提示我：花瓣相关特征很可能是后续分类的关键。

为了进一步看清不同模型“到底是怎么分的”，我又把二分类任务下几种代表性模型的决策边界画了出来。

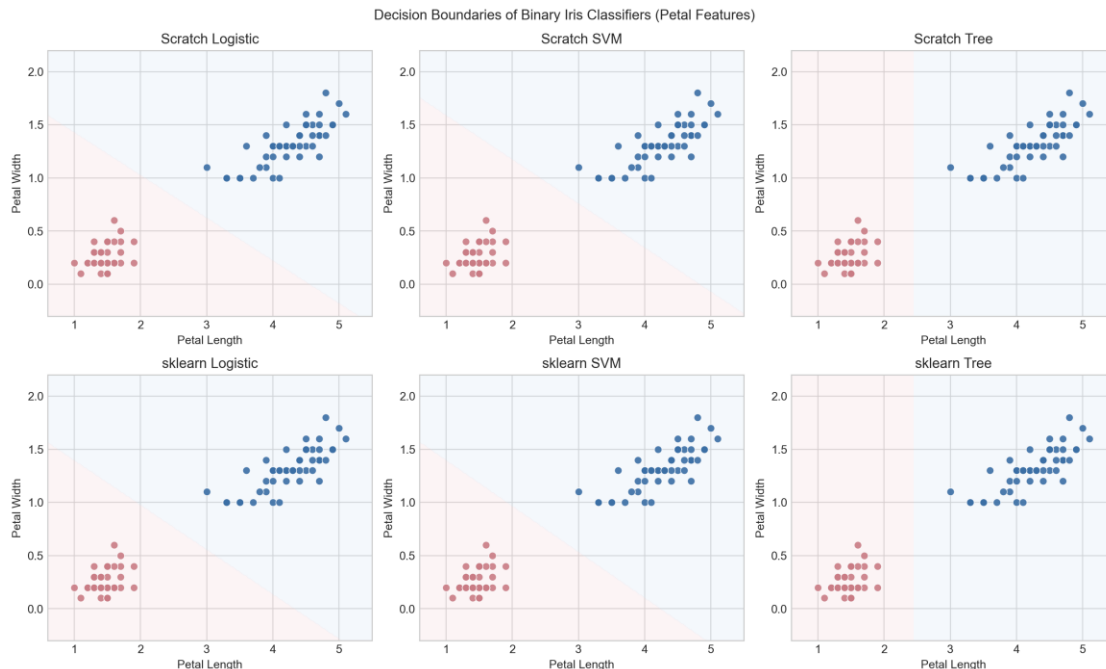


图 3-17 二分类任务下不同模型的决策边界图

这张图很好地体现了模型家族之间的差异。逻辑回归和线性 SVM 的边界基本都是平滑直线，只是位置略有区别，这和它们本身的线性判别假设是一致的；决策树的边界则呈现出明显的分段矩形结构，更像按照阈值逐步切割平面。手写版和 sklearn 版在边界形状上非常接近，这也从可视化角度再次说明，我写出来的模型在核心行为上是和成熟实现一致的。

同样的思路放到原始三分类任务上，模型之间的差异会更明显。



图 3-18 三分类任务下代表性模型的决策区域图

在三分类决策区域里，**Scratch OvR Logistic** 给出的还是相对平滑的线性分区；**Scratch Tree** 会形成更明显的轴对齐切分；**Gaussian Naive Bayes** 的边界相对光滑，但会受到分布假设的影响；**Random Forest** 则在局部边界上更灵活，能够给出更细致的非线性划分。这张图让我更容易把前面的准确率结果和模型结构对应起来，因为它把“线性模型为什么容易受限”“树模型为什么更灵活”这些概念直接画了出来。

3.14 线性判别能力与系数分析

在二分类任务中，我还对代表性模型补了 ROC 曲线。这个分析的重点不是看某个固定阈值下分得对不对，而是比较模型在不同阈值下整体的判别能力。

表 3-8 二分类任务 ROC AUC 结果

模型	AUC
Scratch Logistic	1.0000
Scratch SVM	1.0000
sklearn Logistic	1.0000
sklearn SVM	1.0000

Gaussian Naive Bayes 1.0000
Random Forest 1.0000

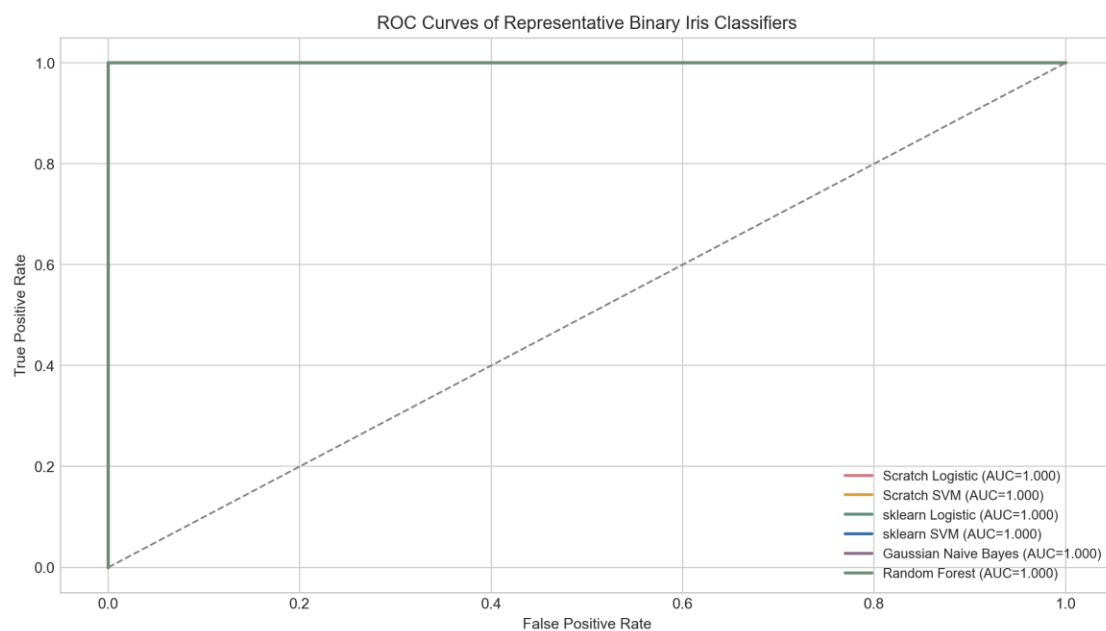


图 3-19 二分类任务下代表性模型 ROC 曲线图

这组 ROC 结果有一个很鲜明的特点，就是几种代表性方法的 AUC 都达到了 1.0000。从好的方面看，这说明当前二分类 Iris 任务确实非常容易，模型即使换了不同家族，整体判别能力依然都很强；但从分析角度看，这也再次提醒我，在过于简单的数据集上，单一指标很容易“全部满分”，真正能拉开差距的往往不是最终分数，而是效率、边界形状、鲁棒性和概率质量这些更细的维度。

线性模型还有一个很适合补充分析的地方，就是参数本身具有比较直接的可解释性。把逻辑回归和线性 SVM 的系数画出来之后，可以更直观看到它们更依赖哪些特征。



图 3-20 线性模型特征系数对比图

从系数图来看，不管是手写版还是 sklearn 版，逻辑回归和线性 SVM 都表现出相似的趋势：**Petal Length** 和 **Petal Width** 的系数明显为正，而且幅度更大；**Sepal Width** 则是明显负向的。这个现象说明，在当前二分类任务中，花瓣长度和花瓣宽度的增大更容易把样本推向后一类，而萼片宽度则在相反方向上起作用。虽然不同实现的系数绝对值不一样，但方向和相对重要性是一致的，这也说明这些线性模型确实抓住了同一组关键区分信息。

3.15 集成模型特征重要性分析

对于树模型和集成模型来说，参数本身不像线性模型那样直接，但它们会在训练过程中自然给出特征重要性。因此我又把随机森林和梯度提升树的特征重要性提取出来做了比较。

表 3-9 集成模型特征重要性结果

特征	Random Forest	Gradient Boosting
Sepal Length	0.1042	0.0040
Sepal Width	0.0153	0.0042
Petal Length	0.4378	0.4506
Petal Width	0.4427	0.5412

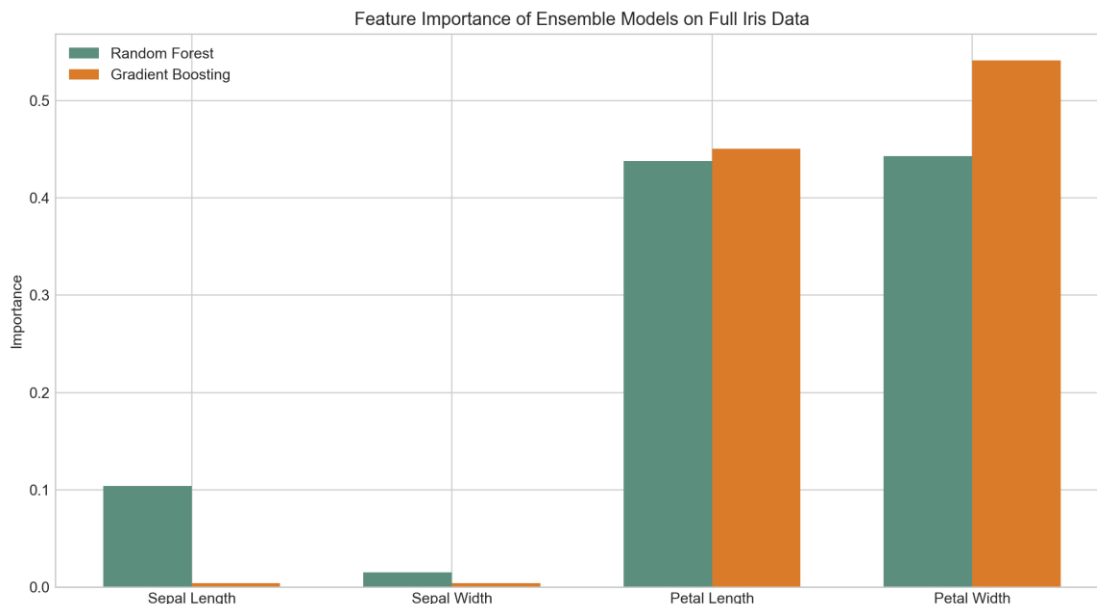


图 3-21 集成模型特征重要性对比图

这张图和前面的特征消融实验其实是相互印证的。无论是随机森林还是梯度提升树，都把主要权重放在了 **Petal Length** 和 **Petal Width** 上，而 **Sepal Width** 的重要性非常低。尤其是梯度提升树，对 **Petal Width** 的依赖最强，重要性达到 **0.5412**。这说明不只是单一模型会偏向花瓣特征，连基于树的集成方法也在反复利用同一组关键信息进行分裂和修正。对我来说，这样的结果会比单纯说“花瓣特征更重要”更有说服力，因为它已经在不同模型、不同分析方式之间形成了相互支持的证据链。

4. 实验总结

如果从整体结果来概括这次实验，可以得到一个比较清楚的认识：在当前这份 **Iris** 数据上，很多模型都能够取得较高准确率，但这并不意味着它们在训练效率、稳定性、鲁棒性、样本效率、概率可靠性和边界形状上也同样接近。随着实验维度逐步扩展，这些差异反而变得比单纯的准确率更值得分析。

通过本实验可以较清楚地看到，手写实现与成熟库实现各有不同价值。手写模型更适合用于理解算法的核心机制，例如逻辑回归中的概率建模、**SVM** 中的最大间隔思想，以及决策树中的递归划分过程；而成熟库实现则在训练效率、数值稳定性和工程可用性上具有明显优势。

扩展实验进一步说明，模型比较不能只停留在单一准确率指标上。在原始三分类任务、噪声扰动测试、特征消融分析、深度学习实现方式比较，以及新增的数据分布、决策边界、**ROC** 曲线和特征重要性分析里，不同模型在稳定性、鲁棒性、训练效率、复杂度和可解释性方面的差异都表现得比较明显。尤其是样本效率实验显示，在小样本表格任务上，传统方法通常更早进入稳定区间；概率校准实验则说明，神经网络即使分类结果不差，也可能给出过度自信的概率输出；而决策区域和

特征重要性图又进一步把这些差异对应回了模型结构本身。因此，模型选择不仅要考虑精度，还需要结合数据规模、任务风险、输出形式和可解释需求综合判断。

从研究角度看，这次实验得到的现象与近年的表格学习文献是能够相互印证的。一方面，深度学习在表格任务上并没有天然的普适优势；另一方面，深度模型在更复杂特征交互、更大规模样本或者更丰富先验结构下依然可能展现出更强潜力。也就是说，这里的结论并不是“传统模型一定更好”或者“深度学习没有意义”，而是要根据任务特征选择合适的方法，并且使用足够全面的评价指标去理解模型行为。

如果后续继续完善本实验，我还想在更大规模的数据集上引入 TabTransformer、FT-Transformer 或残差型表格网络，与随机森林和梯度提升树继续做统一协议下的比较；也想对手写模型继续增加剪枝、正则化和更稳定的多分类策略，并进一步考察校准方法在不同模型上的通用性。这样做之后，这个实验会更接近一个小型研究项目。

总体来看，本实验不仅完成了基础分类模型的实现与比较，也通过多方法扩展、多维度分析以及研究型补充实验，使结果具有了更完整的解释力。相比只展示几个模型的最终准确率，这种从效率、稳定性、鲁棒性、样本效率和概率可靠性多角度展开的分析，更能够真实反映不同方法在表格分类任务中的特点。

5. 参考文献

[1] Grinsztajn, L., Oyallon, E., and Varoquaux, G. Why do tree-based models still outperform deep learning on tabular data? arXiv:2207.08815, 2022.
<https://arxiv.org/abs/2207.08815>

[2] Gorishniy, Y., Rubachev, I., Khrulkov, V., and Babenko, A. Revisiting Deep Learning Models for Tabular Data. arXiv:2106.11959, 2021.
<https://arxiv.org/abs/2106.11959>

[3] Huang, X., Khetan, A., Cvitkovic, M., and Karnin, Z. TabTransformer: Tabular Data Modeling Using Contextual Embeddings. arXiv:2012.06678, 2020.
<https://arxiv.org/abs/2012.06678>

[4] Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On Calibration of Modern Neural Networks. Proceedings of the 34th International Conference on Machine Learning, 2017. <https://cit.rii.ac.jp/crid/1370861707377531922?lang=en>

[5] Hastie, T., Tibshirani, R., and Friedman, J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition. Springer, 2009.

[6] Bishop, C. M. Pattern Recognition and Machine Learning. Springer, 2006.

[7] Cortes, C. and Vapnik, V. Support-Vector Networks. Machine Learning, 20(3):273-297, 1995.

[8] Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. Classification and Regression Trees. Wadsworth, 1984.

[9] Breiman, L. Random Forests. *Machine Learning*, 45(1):5-32, 2001.

[10] Freund, Y. and Schapire, R. E. A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1):119-139, 1997.

[11] Friedman, J. H. Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5):1189-1232, 2001.

[12] Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT Press, 2016.